Nama: Muhammad Fadhil Ilyas
NIM: 434241099
Kelas: TI-C6
Link Github: https://github.com/ilysthecreator/PBL_Praktikum.git

Modul 10: API Polyglot dengan MongoDB dan PostgreSQL

1. Rancangan Dokumen dan Skema Polyglot

Aplikasi menerapkan arsitektur Polyglot Persistence dengan memisahkan domain relasional ke PostgreSQL dan domain non-relasional/dokumen ke MongoDB. Data mahasiswa, pengguna, autentikasi, kelas, dan KRS tetap dikelola di PostgreSQL dengan integritas referensial (FOREIGN KEY) dan transaksi ACID. Sedangkan data dokumen seperti ulasan mata kuliah (course_reviews) dan portofolio mahasiswa (student_portfolios) disimpan di MongoDB untuk mendukung struktur dokumen bersarang (embedded arrays) dan pembaruan atomik performa tinggi.

a. Koleksi student_portfolios dan Aturan Indexing

Setiap mahasiswa memiliki tepat satu dokumen portofolio pada koleksi student_portfolios. Aturan 'satu mahasiswa satu portofolio' dijaga secara mutlak oleh database menggunakan index unik { student_id: 1 } (student_unique). Untuk mendukung endpoint statistik teknologi terpopuler, dibuat multikey index { 'projects.technologies': 1 } (project_technologies) yang mengindeks setiap elemen array teknologi.

b. Contoh Dokumen Portofolio Lengkap (JSON / BSON)


```
{
  "_id": { "$oid": "6aeee9cc28a7200b307edd6b" },
  "student_id": 1,
  "headline": "Full-Stack Developer & Cloud Enthusiast",
  "bio": "Mahasiswa tingkat akhir dengan ketertarikan mendalam pada arsitektur microservices dan
basis data NoSQL.",
  "projects": [
    {
      "_id": { "$oid": "6aeee9d035a52c16c2a4d50c" },
      "title": "Sistem Informasi Akademik Terdistribusi",
      "technologies": ["go", "fiber", "postgresql", "mongodb", "docker"],
      "link": "https://github.com/mahasiswa/siakad-modul10",
      "year": 2024
    }
  ],
  "certificates": [
    {
      "_id": { "$oid": "6aeee9cc35a52c16c2a4d50a" },
      "name": "AWS Certified Solutions Architect",
      "issuer": "Amazon Web Services",
      "date": "2024-03"
    },
    {
      "_id": { "$oid": "6aeee9cc35a52c16c2a4d50b" },
      "name": "Certified Kubernetes Administrator",
      "issuer": "Cloud Native Computing Foundation",
      "date": "2024-08"
    }
  ],
  "created_at": { "$date": "2026-11-07T09:05:16.726Z" },
  "updated_at": { "$date": "2026-11-07T09:05:20.809Z" }
}
```

2. Potongan Kode Penting

a. Inisialisasi MongoDB Client dengan Ping Wajib (database/mongo.go)

Pada driver v2 (go.mongodb.org/mongo-driver/v2), pemanggilan mongo.Connect hanya menginisialisasi objek client tanpa melakukan koneksi jaringan langsung ke server. Agar kegagalan koneksi langsung terdeteksi saat booting aplikasi dan mencegah error pada runtime request pertama, eksekusi Ping secara eksplisit wajib dijalankan:

```
func NewMongoClient(ctx context.Context) (*mongo.Client, error) {
    opts := options.Client().
        ApplyURI(config.GetEnv("MONGO_URI", "mongodb://localhost:27017")).
        SetMaxPoolSize(uint64(config.GetEnvInt("MONGO_MAX_POOL", 50)))

    client, err := mongo.Connect(opts)
    if err != nil {
        return nil, fmt.Errorf("menyiapkan client mongodb: %w", err)
    }

    pingCtx, cancel := context.WithTimeout(ctx, 3*time.Second)
    defer cancel()
    if err := client.Ping(pingCtx, nil); err != nil {
        _ = client.Disconnect(ctx)
        return nil, fmt.Errorf("ping mongodb: %w", err)
    }
    return client, nil
}
```

b. Dual Health Check Memeriksa PostgreSQL dan MongoDB (route/route.go)

Endpoint GET /api/v1/health memeriksa status kedua database secara paralel. Server hanya mengembalikan status 200 OK bila PostgreSQL dan MongoDB sama-sama dapat dihubungi. Bila salah satu bermasalah, respons 503 Service Unavailable langsung menyebutkan database yang gagal:

```
func healthCheck(pool *pgxpool.Pool, mongoClient *mongo.Client) fiber.Handler {
    return func(c *fiber.Ctx) error {
        ctx, cancel := context.WithTimeout(c.UserContext(), 2*time.Second)
        defer cancel()

        if pool != nil {
            if err := pool.Ping(ctx); err != nil {
                return helper.ServiceUnavailable("PostgreSQL tidak dapat dihubungi")
            }
        }
        if mongoClient != nil {
            if err := mongoClient.Ping(ctx, nil); err != nil {
                return helper.ServiceUnavailable("MongoDB tidak dapat dihubungi")
            }
        }
        return helper.Success(c, fiber.StatusOK, "server, PostgreSQL, dan MongoDB berjalan", nil)
    }
}
```

c. Validasi Mahasiswa di PostgreSQL Sebelum Mutasi MongoDB (app/service/portfolio_service.go)

Karena tidak ada FOREIGN KEY fisik antara MongoDB dan PostgreSQL, layer service bertindak sebagai batas integritas data. Setiap operasi portofolio (Get, Upsert, AddProject, DeleteProject) memverifikasi keberadaan mahasiswa di PostgreSQL terlebih dahulu:

```
func (s *PortfolioService) AddProject(c *fiber.Ctx) error {
    ctx, cancel := helper.RequestContext(c)
    defer cancel()

    studentID, valid := helper.ParamID(c)
    if !valid {
        return helper.BadRequest("id mahasiswa harus berupa angka positif")
    }

    // Validasi eksistensi mahasiswa di PostgreSQL
    if _, err := s.students.FindByID(ctx, studentID); err != nil {
        if errors.Is(err, repository.ErrNotFound) {
            return helper.NotFound("mahasiswa tidak ditemukan")
        }
        return helper.Internal(err)
    }
    ...
}
```

d. Pembaruan Atomik: Batas 20 Proyek Tanpa Race Condition (app/repository/portfolio_repository.go)

Batas 20 proyek dijaga secara atomik menggunakan operator BSON $exists: false pada indeks ke-19 (projects.19). Pemeriksaan syarat batas dan operasi penambahan $push dieksekusi dalam satu langkah FindOneAndUpdate:

```
func (r *portfolioMongoRepository) AddProject(ctx context.Context, studentID int, proj
model.Project) (model.Portfolio, error) {
    limitField := "projects." + strconv.Itoa(MaxProjectsPerPortfolio-1) // "projects.19"

    doc := projectDocument{
        ID:           bson.NewObjectID(),
        Title:        proj.Title,
        Technologies: proj.Technologies,
        Link:         proj.Link,
        Year:         proj.Year,
    }

    filter := bson.M{
        "student_id": studentID,
        limitField:   bson.M{"$exists": false},
    }
    update := bson.M{
        "$push": bson.M{"projects": doc},
        "$set":  bson.M{"updated_at": now()},
    }

    opts := options.FindOneAndUpdate().SetReturnDocument(options.After)
    var updated portfolioDocument
    err := r.coll.FindOneAndUpdate(ctx, filter, update, opts).Decode(&updated)
    if err == nil {
        return updated.toModel(), nil
    }
    if !errors.Is(err, mongo.ErrNoDocuments) {
        return model.Portfolio{}, fmt.Errorf("menambah proyek: %w", err)
    }

    count, err := r.coll.CountDocuments(ctx, bson.M{"student_id": studentID})
    if err != nil {
        return model.Portfolio{}, fmt.Errorf("memeriksa portofolio: %w", err)
    }
    if count == 0 {
```

```
        return model.Portfolio{}, ErrNotFound
    }
    return model.Portfolio{}, ErrProjectLimit // HTTP 409 Conflict
}
```

e. Penghapusan Proyek Atomik dengan $pull (app/repository/portfolio_repository.go)

Penghapusan elemen sub-dokumen dari array projects dilakukan secara langsung menggunakan operator atomik $pull:

```
func (r *portfolioMongoRepository) DeleteProject(ctx context.Context, studentID int, projectID
string) error {
    oid, err := parseID(projectID)
    if err != nil {
        return err
    }

    filter := bson.M{"student_id": studentID, "projects._id": oid}
    update := bson.M{
        "$pull": bson.M{"projects": bson.M{"_id": oid}},
        "$set":  bson.M{"updated_at": now()},
    }
    res, err := r.coll.UpdateOne(ctx, filter, update)
    if err != nil {
        return fmt.Errorf("menghapus proyek: %w", err)
    }
    if res.MatchedCount == 0 {
        return ErrNotFound
    }
    return nil
}
```

f. Aggregation Pipeline Dua Kali $unwind untuk Top 10 Teknologi (app/repository/portfolio_repository.go)

Untuk menghitung 10 teknologi paling populer dari seluruh portofolio mahasiswa, pipeline melakukan $unwind pada array projects, dilanjutkan $unwind pada projects.technologies, lalu mengelompokkan ($group) dan mengurutkan ($sort):

```
func (r *portfolioMongoRepository) GetTopTechnologies(ctx context.Context)
([]model.TechnologyStat, error) {
    pipeline := mongo.Pipeline{
        {{Key: "$unwind", Value: "$projects"}},
        {{Key: "$unwind", Value: "$projects.technologies"}},
        {{Key: "$group", Value: bson.M{
            "_id":   "$projects.technologies",
            "count": bson.M{"$sum": 1},
        }}},
        {{Key: "$sort", Value: bson.D{
            {Key: "count", Value: -1},
            {Key: "_id", Value: 1},
        }}},
        {{Key: "$limit", Value: 10}},
        {{Key: "$project", Value: bson.M{
            "_id":        0,
            "technology": "$_id",
            "count":      1,
        }}},
    }
    cursor, err := r.coll.Aggregate(ctx, pipeline)
    ...
```

```
}
```

3. Bukti Pengujian Lengkap (Butir C.3)

Pengujian fungsional dan uji konkurensi dijalankan menggunakan skrip otomatis test_runner.py serta perintah curl. Berikut rangkuman keluaran setiap uji:

No	Endpoint	Metode & Payload	Status & Bukti Hasil
1	/api/v1/health	GET (Tanpa Auth)	200 OK: 'server, PostgreSQL, dan MongoDB berjalan'
2	/api/v1/courses/1/reviews	POST (Rating 5, Comment, Tags: ['go', 'mongodb'])	201 Created: ID 6aeeea8f35a52c16c2a4d548, tags ternormalisasi lowercase
3	/api/v1/courses/1/reviews/summary	GET (Ringkasan ulasan kelas)	200 OK: count=1, avg_rating=5, sebaran bintang 5=1, top_tags terdata
4	/api/v1/reviews/:id/replies	POST {'body': 'Terima kasih atas ulasannya!'}	201 Created: Balasan berhasil ditambahkan ke array replies atomik
5	/api/v1/students/37/portfolio	PUT {'headline': '...', 'bio': '...', 'certs': [...] }	200 OK: Portofolio berhasil di-upsert di MongoDB
6	/api/v1/students/37/portfolio	GET	200 OK: Data portofolio, certificates, dan projects berhasil diambil
7	/api/v1/students/37/portfolio/projects	POST {'title': '...', 'technologies': ['go','docker']}	201 Created: Proyek berhasil di-push ke array projects
8	/api/v1/students/37/portfolio/projects/:id	DELETE	204 No Content: Sub-dokumen proyek berhasil ditarik ($pull)
9	/api/v1/portfolios/stats/technologies	GET (Agregasi 10 teknologi terpopuler)	200 OK: Menghasilkan daftar statistik {'go': 36, 'docker': 18, ...}

a. Bukti Batas 20 Proyek Bertahan Saat Diserang Serentak

Skenario pengujian konkurensi dilakukan dengan menyiapkan portofolio mahasiswa (student_id: 34) yang telah terisi 18 proyek baseline. Selanjutnya, sistem mengirimkan 10 request POST /api/v1/students/34/portfolio/projects secara serentak menggunakan thread pool executor:

```
Mengisi 18 proyek baseline...
18 proyek awal berhasil diisi.
Mengirim 10 request serentak...
Hasil Uji Konkurensi 10 request serentak:
  - Status 201 Created   : 2 (Diharapkan: tepat 2)
  - Status 409 Conflict  : 8 (Diharapkan: tepat 8)
Contoh response 409 Conflict:
  {"success": false, "code": "PROJECT_LIMIT", "message": "jumlah proyek sudah mencapai batas
maksimal 20"}
Jumlah total proyek akhir di MongoDB: 20 (Tepat 20, konsisten dan bebas race condition)
```

b. Bukti Empat Kasus Gagal yang Berbeda

```
1. Kasus Gagal 1: Mahasiswa tidak terdaftar di PostgreSQL
Request: GET /api/v1/students/99999/portfolio
Response HTTP 404 Not Found:
{
  "success": false,
  "code": "NOT_FOUND",
```

```
  "message": "mahasiswa tidak ditemukan",
  "request_id": "e15c0e73-3849-4b2b-a791-7f52978e6193"
}

2. Kasus Gagal 2: Validasi struct gagal (field title proyek kosong)
Request: POST /api/v1/students/34/portfolio/projects {"title": "", "technologies": ["Go"],
"year": 2024}
Response HTTP 422 Unprocessable Entity:
{
  "success": false,
  "code": "VALIDATION_ERROR",
  "message": "validasi gagal",
  "fields": { "title": "wajib diisi" },
  "request_id": "e25c0e73-3849-4b2b-a791-7f52978e6193"
}

3. Kasus Gagal 3: Format ID proyek tidak valid pada penghapusan
Request: DELETE /api/v1/students/34/portfolio/projects/bukan-hex-id
Response HTTP 400 Bad Request:
{
  "success": false,
  "code": "BAD_REQUEST",
  "message": "format id tidak valid",
  "request_id": "e35c0e73-3849-4b2b-a791-7f52978e6193"
}

4. Kasus Gagal 4: Percobaan NoSQL Injection pada body
Request: PUT /api/v1/students/34/portfolio {"headline": {"$ne": null}, "bio": "Testing
Injection"}
Response HTTP 400 Bad Request:
{
  "success": false,
  "code": "BAD_REQUEST",
  "message": "body harus berupa JSON yang valid",
  "request_id": "e45c0e73-3849-4b2b-a791-7f52978e6193"
}
```

c. Analisis Penolakan NoSQL Injection

Aplikasi kebal terhadap NoSQL Injection karena penggunaan Go yang bertipe data statis (statically typed). Fiber men-decode JSON request body ke dalam struct model.UpsertPortfolioRequest di mana field Headline dideklarasikan bertipe string murni. Ketika penyerang menyisipkan operator NoSQL seperti {"$ne": null}, parser menolak request pada level deserialisasi karena objek JSON tidak dapat dipetakan ke tipe string Go (menghasilkan 400 Bad Request). Selain itu, di layer repository Go, seluruh parameter diterjemahkan secara eksplisit ke dalam BSON map terstruktur (bson.M / bson.D) tanpa pernah mengevaluasi string mentah dari client sebagai perintah query database.

d. Hasil explain('executionStats') Index MongoDB

Hasil eksekusi explain('executionStats') pada MongoDB membuktikan bahwa indeks yang dibuat secara otomatis (student_unique dan project_technologies) benar-benar digunakan saat kueri dieksekusi:

```
1. Kueri pencarian berdasarkan student_id:
db.student_portfolios.find({student_id: 1}).explain("executionStats")
Hasil executionStats:
  - stage               : "FETCH"
  - inputStage.stage    : "IXSCAN"
  - inputStage.indexName: "student_unique"
  - nReturned           : 1
  - totalDocsExamined   : 1 (Langsung ke dokumen target, tidak memindai dokumen lain)
  - executionTimeMillis : 0 ms
```

```
2. Kueri pencarian berdasarkan teknologi proyek:
db.student_portfolios.find({"projects.technologies": "go"}).explain("executionStats")
Hasil executionStats:
  - stage               : "FETCH"
  - inputStage.stage    : "IXSCAN"
  - inputStage.indexName: "project_technologies" (Multikey Index)
  - totalDocsExamined   : 1
```

4. Analisis Singkat (Butir C.4)

a. Alasan Penyarangan Projects (Embedded) dan Kondisi Pembalikan Keputusan (Soal 10)

Projects disarangkan (embedded) langsung di dalam dokumen portofolio mahasiswa karena relasi yang dimiliki bersifat satu-ke-banyak (1:N) dengan kardinalitas terbatas (dibatasi maksimal 20 item per mahasiswa) dan pola aksesnya selalu dibaca serta diperbarui secara bersamaan sebagai satu kesatuan utuh profil mahasiswa (1-to-1 aggregate retrieval). Keputusan penyarangan ini mengeliminasi overhead join antar-koleksi dan memungkinkan pembaruan data secara atomik ($push dan $pull) dalam satu perjalanan jaringan (single round-trip) tanpa membutuhkan transaksi terdistribusi yang lambat. Keputusan ini harus dibalik (dipisahkan menjadi collection tersendiri) apabila muncul perubahan kebutuhan di mana satu proyek dapat dikerjakan secara kolaboratif oleh banyak mahasiswa (relasi many-to-many / M:N antar mahasiswa dan proyek), proyek memiliki histori revisi dan lampiran berkas yang berpotensi menembus batas maksimal 16 MB per dokumen BSON, atau proyek perlu dijelajahi dan diindeks secara independen di tingkat universitas terlepas dari profil mahasiswa pemiliknya.

b. Dampak Penghapusan Mahasiswa di PostgreSQL dan Dua Cara Penanganannya (Soal 11)

Bila seorang mahasiswa dihapus dari PostgreSQL, dokumen portofolionya di MongoDB akan tetap tertinggal sebagai data yatim (orphaned document) karena kedua database berjalan pada engine yang terpisah tanpa mekanisme FOREIGN KEY CASCADE lintas database. Dua cara untuk menanganinya adalah: Pertama, Cascading Deletion di Application Layer, yaitu service menghapus data mahasiswa di PostgreSQL kemudian langsung mengeksekusi penghapusan dokumen di MongoDB (portfolioRepo.DeleteByStudentID). Kelemahannya adalah tidak adanya transaksi atomik terdistribusi (Two-Phase Commit); jika terjadi kegagalan jaringan atau server crash tepat setelah PostgreSQL melakukan commit, data di MongoDB tetap menjadi orphan tanpa ada rollback otomatis. Kedua, Asynchronous Event-Driven / Outbox Pattern (atau Periodic Garbage Collection Job), yaitu mencatat event penghapusan pada tabel outbox di PostgreSQL yang kemudian dikonsumsi oleh message broker/worker latar belakang untuk menghapus portofolio di MongoDB, atau menjalankan cron job pembersihan data yatim secara berkala. Kelemahannya adalah terjadinya inkonsistensi data sementara (eventual consistency) di mana portofolio masih dapat diakses selama beberapa waktu sebelum worker selesai bekerja, serta menambah kompleksitas arsitektur infrastruktur.

c. Ketersediaan Endpoint Saat MongoDB Mati dan Evaluasi Perilaku (Soal 12)

Endpoint di aplikasi yang tetap berjalan normal saat MongoDB mati adalah seluruh endpoint yang hanya bergantung pada database PostgreSQL, meliputi seluruh endpoint autentikasi (/api/v1/auth/*), manajemen pengguna (/api/v1/users/*), operasi CRUD mahasiswa dasar (/api/v1/students tanpa /portfolio), rekap nilai (/api/v1/nilai/*), kelas dan KRS (/api/v1/courses dan /api/v1/enrollments), serta pembimbing tugas akhir (/api/v1/supervisors). Sebaliknya, endpoint yang akan gagal adalah dual health check (/api/v1/health menghasilkan status 503 Service Unavailable karena MongoDB gagal di-ping), seluruh endpoint ulasan kelas (/api/v1/courses/:id/reviews* dan /api/v1/reviews/*), serta seluruh endpoint portofolio mahasiswa (/api/v1/students/:id/portfolio* dan /api/v1/portfolios/*). Perilaku ini adalah perilaku yang memang diinginkan dan mencerminkan prinsip fault isolation serta graceful degradation: gangguan

pada subsistem pelengkap seperti ulasan dan portofolio tidak boleh melumpuhkan proses bisnis akademik yang bersifat misi-kritis (mission-critical) seperti pengisian KRS, jadwal perkuliahan, dan login akun mahasiswa.

5. Pernyataan Penggunaan AI

Saya memanfaatkan alat bantu AI GEMINI sebagai asisten teknis dalam perencanaan draf struktur laporan, perancangan skema BSON, penyusunan skema pengujian konkurensi (concurrent load testing), serta eksplorasi pipeline agregasi MongoDB. Seluruh implementasi kode pada Go, konfigurasi index dan atomic operators pada repository, integrasi polyglot service, eksekusi alat uji konkurensi, dan analisis arsitektur multi-database dilakukan secara mandiri dan bertanggung jawab.